

PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS

GABRIEL HENRIQUE MOTA RODRIGUES - 814411

TEST PATTERN

BELO HORIZONTE - 2025

1. Padrões de Criação de Dados (Builders)

Por que o CarrinhoBuilder foi usado em vez de um CarrinhoMother

O padrão Builder foi utilizado no lugar de um Mother Object porque o objetivo era criar instâncias de Carrinho de forma flexível e reutilizável, evitando o acoplamento a configurações fixas.

O CarrinhoMother normalmente cria objetos prontos e específicos, como CarrinhoMother.umCarrinhoComUmProduto(). Esse padrão é útil para casos simples, mas se torna limitado e de difícil manutenção quando há muitas variações de dados ou quando o objeto possui muitos atributos opcionais.

Já o CarrinhoBuilder permite uma composição fluida, em que cada teste pode configurar apenas o que é necessário:

Antes:

```
const cliente = new Cliente("Gabriel", "Premium");
const produtos = [
  new Produto("Cerveja", 6, 10.0),
  new Produto("Refrigerante", 2, 5.0)
];
const carrinho = new Carrinho(cliente, produtos, 10);
```

Depois:

```
const carrinho = new CarrinhoBuilder()
  .comClientePremium("Gabriel")
  .comProdutos([ { nome: "Cerveja", qtd: 6, preco: 10 } ])
  .comDesconto(10)
  .build();
```

Justificativa: legibilidade e manutenção

O uso do Builder melhora a legibilidade ao evidenciar a intenção do teste (“um carrinho com cliente Premium e desconto de 10%”).

Além disso, quando a estrutura da classe Carrinho muda, apenas o Builder precisa ser atualizado, não todos os testes que o utilizam.

Isso reduz o custo de manutenção e previne o Test Smell de Data Clump, onde múltiplos parâmetros e instâncias se repetem em vários testes.

2. Padrões de Test Doubles (Mocks vs. Stubs)

Teste de sucesso “Premium” (Etapa 5)

No teste de “sucesso Premium”, o fluxo verificava se um cliente Premium tinha seu pagamento aprovado e recebia um e-mail de confirmação após o processamento do pedido.

```
test

it("deve processar pedido de cliente Premium e enviar e-mail", async () => {
  const gateway = new GatewayPagamentoStub(); // Stub
  const emailService = new EmailServiceMock(); // Mock

  const useCase = new ProcessarPedido(gateway, emailService);
  const pedido = new PedidoBuilder().comClientePremium().build();

  await useCase.executar(pedido);

  expect(emailService.foiEnviadoPara(pedido.cliente.email)).toBeTruthy();
});
```

Qual dependência foi usada como Stub e qual foi Mock

- **GatewayPagamento: Stub**

O GatewayPagamento foi implementado como um Stub porque seu comportamento foi pré-programado para simular uma resposta (“pagamento aprovado”) sem realizar chamadas reais.

O teste apenas verifica o estado resultante (pedido aprovado), sem checar se métodos do gateway foram chamados. Isso caracteriza uma verificação de estado (State Verification).

- **EmailService: Mock**

Já o EmailService foi implementado como um Mock, pois o objetivo do teste era verificar se e como o método de envio de e-mail foi chamado.

Assim, o foco está na verificação de comportamento (Behavior Verification), algo típico do uso de Mocks.

3. Conclusão

O uso de padrões de teste como Builders e Test Doubles representa uma prática essencial para garantir a clareza, a confiabilidade e a manutenção de uma suíte de testes. O padrão Builder contribui diretamente para a organização e legibilidade do código de teste, permitindo a criação de dados de forma expressiva e reutilizável. Com ele, evita-se a duplicação de código e facilita-se a adaptação a mudanças no modelo de domínio, já que as modificações se concentram em um único ponto — o próprio Builder.

Por outro lado, o uso de Test Doubles, especialmente Mocks e Stubs, permite isolar dependências externas, como gateways de pagamento, repositórios e serviços de e-mail, garantindo que os testes sejam rápidos, determinísticos e independentes de integrações reais. O Stub é empregado quando o foco está na verificação de estado, simulando respostas previsíveis sem testar comportamento, enquanto o Mock é usado quando o objetivo é verificar interações e garantir que determinados métodos foram chamados conforme o esperado.

A aplicação conjunta desses padrões reduz a ocorrência de Test Smells, como testes obscuros, dependentes de ambiente ou de difícil manutenção, além de tornar o código de teste mais expressivo e alinhado à intenção do caso de uso. Em suma, o uso consciente de Builders e Test Doubles contribui para uma suíte de testes sustentável, robusta e alinhada às boas práticas de engenharia de software, fortalecendo a qualidade geral do sistema ao longo de seu ciclo de vida.